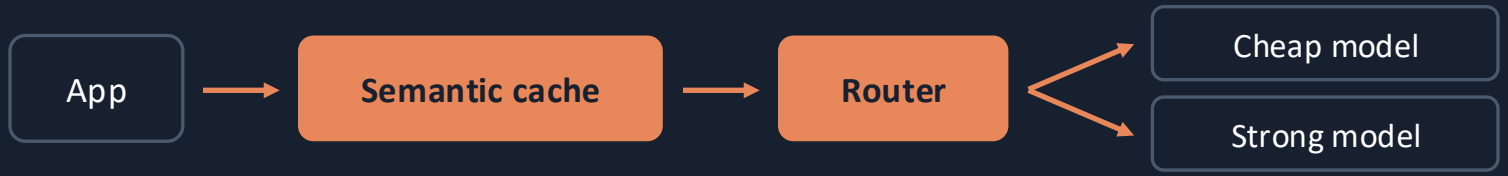
EC601 · SPRINT 1 · TEAM CACHESHIFT

CacheShift

What shifts when a semantic cache sits in front of an LLM router?



Spencer Aldrich · Anthony Capraru

github.com/cacheshift-project/cacheshift

Mission Statement

For engineers running an LLM feature on a budget, with a cache in front of a model router, CacheShift is a gateway and measurement tool that shows how the cache shifts the router's cost and answer quality, and re-tunes the router for the questions it actually sees. Unlike dashboards that only count cache hits and savings, it shows how good the cached answers are and how far the router has drifted, with margins of error.

Changed since the plan

Story 1: "does my budget hold?" → "is my cost/quality trade-off still where I set it?"

Why

Fixed cutoff + cache = fewer calls, so spend can't go over. The real risk is quality and cost per question drifting.

Sprint 1 Progress

#	Story	Done so far:
1	See the drift: planned vs. real strong-model share	First controlled result
2	Check every cached answer against the right answer	Loose matching serves up to 3% wrong-question answers. Hand-check pending
3	One command re-tunes the router after the cache	Misses on ARC; on target on MMLU, but no drift there to fix
4	Point my app at the gateway by changing one URL	Works in replay mode
5	A test set with human-written answer keys	Official keys joined; no rewordings yet

Tools We Are Using

Python 3.12
ML tools live there

FastAPI
OpenAI-style endpoint, so apps switch by URL

RouteLLM (BERT)
The router we study; runs on a laptop CPU

RouterBench
11 models' recorded answers: replay costs $0

pandas + NumPy
Metrics and bootstrap confidence intervals

pytest + Actions
Tests on every PR; nobody pushes to main

Claude + ChatGPT
Two-model plan review; chats logged in repo

PROGRESS · WHAT RUNS TODAY

Cache → router → answer, end to end

36,497

questions × 11 models loaded from RouterBench, safety-checked

0 GPUs

RouteLLM router scores questions on a laptop CPU, no API key

16 papers

Key insight: RouteLLM sets its cutoff as a percentile of a fixed sample, so a cache can't flip one decision, only shift the overall share

```
CacheShift | End-to-end replay demo

Mode          Requests  Hits  Routed  Strong
no_cache            15     0      15      10
exact_cache         15     5      10       8

Strong share of routed: 66.7% -> 80.0%
Historical cost: $0.008388 -> $0.006699

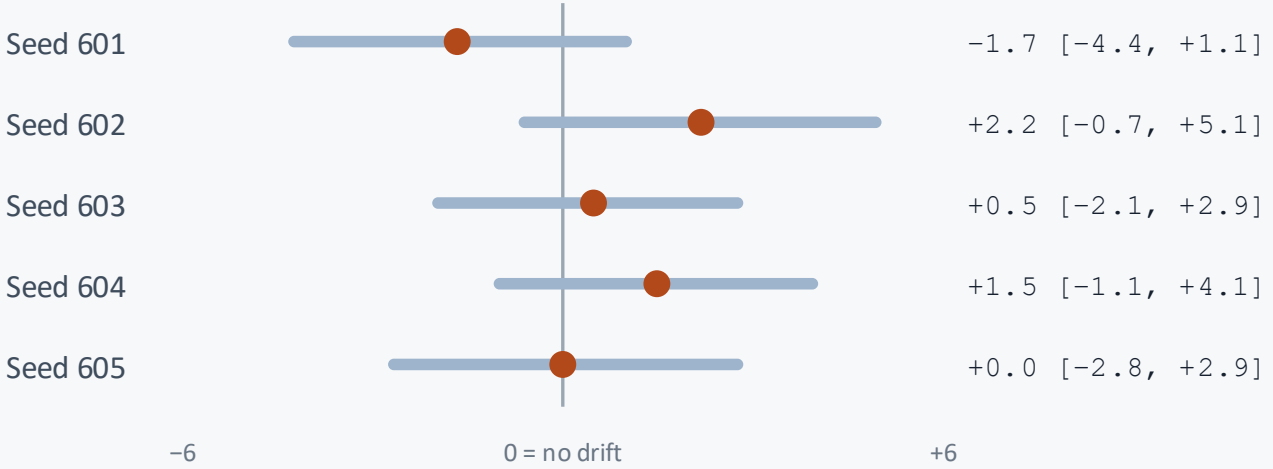
15-request development replay | No paid API calls
```

Replay demo, real output (15 requests). Too small to mean anything; it shows the pipe works and logs every request.

WHAT WE LEARNED · AN ASSUMPTION OUR TEST CHANGED

Random repeats cause no drift

Cache effect on strong-model share (points), exact cache, 300 ARC questions



Every interval crosses 0

Drift needs the repeated questions to differ from the rest

So A2 changed

Test it on real chat logs: do repeated prompts score differently?

Dot = estimate · bar = 95% bootstrap CI · source: experiments/results/retune-20261004

SPRINT 2

In two weeks we will show **router scores for repeated vs. one-off prompts in real chat logs**, and the gateway running a **semantic cache** that reports planned, no-cache and cached share, each with a 95% CI.

What's in progress:

- Target group: small business owners using AI tools; user conversations remain pending.
- Verified answer keys currently cover ARC and MMLU.
- Retuning corrects synthetic drift, but one scenario loses accuracy; real-traffic validation remains pending.